

ZK-ProVer: Non-Interactive Zero-Knowledge Certification for SAT-Based Program Verification

JINGYU KE, School of Computer Science, Shanghai Jiao Tong University, China

HAOYU WEI, School of Computer Science, Shanghai Jiao Tong University, China

RUIBANG LIU, School of Computer Science, Shanghai Jiao Tong University, China

GUOQIANG LI*, School of Computer Science, Shanghai Jiao Tong University, China

Program verification ensures software correctness through formal methods but often incurs substantial computational overhead. In SAT-based verification, the verification task is reduced to satisfiability checking, where satisfiable instances yield concrete counterexamples and unsatisfiable instances are certified by resolution proofs. While satisfying assignments and resolution proofs are useful for establishing correctness, they may expose defect-relevant details, including concrete inputs that trigger assertion violations, and can be costly for multiple parties to re-check independently. To address this problem, we propose a non-interactive two-phase zero-knowledge protocol for SAT-based program verification that certifies verification results while hiding the satisfying assignment in the SAT case and avoiding transmission of the full resolution proof in the UNSAT case. In Phase I, a zero-knowledge virtual machine (zkVM) performs translation validation for the deterministic frontend-to-CNF translation from the source program and assertions, and binds the resulting SAT formula through a commitment for subsequent verification. In Phase II, we design two specialized AIR constraint systems and implement them over a Plonky3-based STARK backend: one checks satisfying assignments for SAT instances, and the other checks resolution proofs for UNSAT instances, without requiring verifiers to replay the full UNSAT certificate. We evaluate the two phases separately. On supported SV-COMP-style benchmarks, Phase I validates reusable program-to-CNF translations for bounded verification-condition instances. For Phase II, comparison with ZKUNSAT on ten UNSAT instances yields an $11.1\times$ geometric-mean verifier speedup and a $410.4\times$ geometric-mean reduction in verifier-side communication. These component-level results provide evidence for the feasibility of zero-knowledge certification of program-verification results while limiting counterexample disclosure and reducing repeated UNSAT-certificate validation cost.

CCS Concepts: • **Security and privacy** → **Cryptography**; • **Software and its engineering** → **Formal software verification**; • **Theory of computation** → *Program verification*.

Additional Key Words and Phrases: program verification, zero-knowledge proofs, non-interactive proofs, satisfiability checking, zero-knowledge virtual machines

ACM Reference Format:

Jingyu Ke, Haoyu Wei, Ruibang Liu, and Guoqiang Li. 2026. ZK-ProVer: Non-Interactive Zero-Knowledge Certification for SAT-Based Program Verification. 1, 1 (April 2026), 22 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Program verification has become a standard mechanism for establishing strong correctness guarantees in domains such as finance, autonomous systems, and smart contracts [25, 27, 33]. A common verification pipeline reduces the task to

*Corresponding author.

Authors' Contact Information: [Jingyu Ke](#), School of Computer Science, Shanghai Jiao Tong University, Shanghai, China, Windocotber@sjtu.edu.cn; Haoyu Wei, School of Computer Science, Shanghai Jiao Tong University, Shanghai, China, haoyuwei@sjtu.edu.cn; Ruibang Liu, School of Computer Science, Shanghai Jiao Tong University, Shanghai, China, 628628@sjtu.edu.cn; [Guoqiang Li](#), School of Computer Science, Shanghai Jiao Tong University, Shanghai, China, li.g@sjtu.edu.cn.

satisfiability checking: bounded model checking [5], symbolic execution [17], and related bit-vector analyses eventually produce satisfiability modulo theories (SMT) or Boolean satisfiability (SAT) instances whose solver result determines whether an assertion-violating execution exists. If the resulting formula is satisfiable, the solver returns a satisfying assignment that witnesses a concrete counterexample. If it is unsatisfiable, the solver returns a resolution proof, namely a sequence of clause derivations ending in the empty clause, that justifies the absence of such an execution.

These assignments and resolution proofs are often richer than what a verifier should learn. In satisfiable cases, the witness assignment can identify the violated assertion together with the concrete input instance and execution path that trigger it. Such information is highly relevant to defect localization, but it may be inappropriate to disclose in external auditing, coordinated vulnerability disclosure, or outsourced verification workflows. In unsatisfiable cases, the issue is different: the verification result says that no violating execution exists, but the resolution proof may be large enough that transmission and repeated checking become costly. Thus, SAT-based verification naturally creates a selective-disclosure requirement that depends on the solver result: the verifier should learn that a bug exists, or that no bug exists, without necessarily receiving the counterexample assignment or the full resolution proof.

Zero-knowledge proofs (ZKPs) [11] provide a natural basis for such selective disclosure. In principle, one could certify the verification result while hiding the satisfying assignment in the SAT case and replacing the full UNSAT certificate with a compact proof checked by each verifier. The main obstacle is scalability on the UNSAT side. Resolution refutations for SAT formulas generated from program verification tasks can be large, and modeling resolution checking as a generic computation inside a general-purpose zero-knowledge virtual machine (zkVM) does not expose the regular clause-database, pivot, and multiset structure of resolution derivations, thereby introducing substantial prover-time overhead. Interactive zero-knowledge approaches face a complementary limitation: they may reduce some prover-side structure, but they impose higher verifier time and communication cost, which is poorly matched to multi-party, offline, repeated validation.

In this work, we present a non-interactive two-phase zero-knowledge protocol for SAT-based program verification. Phase I uses a zkVM [8] to validate the deterministic frontend-to-CNF translation from the source program and assertions and binds the resulting CNF formula through a cryptographic commitment for later use. Phase II then proves the solver result for the same committed formula using two specialized algebraic intermediate representation (AIR) constraint systems that we design and implement over the Plonky3 STARK (Scalable Transparent Argument of Knowledge) backend [24]. For SAT instances, the resulting proof establishes the existence of a satisfying assignment while hiding the concrete counterexample, including the inputs that identify the failing execution. For UNSAT instances, it establishes resolution validity without requiring the verifier to receive and re-check the full resolution proof. The formula commitment links the two phases, ensuring that the result certified in Phase II is for the SAT formula certified by Phase I.

The resulting verifier interface is deliberately small: it checks two proofs, the public formula commitment, and compact metadata, but receives neither the CNF clauses nor the private witness. This boundary supports both privacy for SAT counterexamples and compact certification for UNSAT proofs.

The main contributions of this paper are:

- We introduce ZK-ProVer, a non-interactive two-phase zero-knowledge certification protocol for SAT-based program verification. It binds a private CNF formula through Phase I program-to-CNF translation validation and certifies the SAT or UNSAT result for that formula while hiding the counterexample assignment in the SAT case and avoiding repeated transmission of the full resolution proof in the UNSAT case.

- We design specialized AIR constraint systems for Phase II over a Plonky3-based STARK backend: the SAT AIR checks clause satisfaction under a private assignment, and the UNSAT AIR validates resolution proofs through ordered clause databases, parent-clause consistency, pivot checks, and multiset constraints.
- We implement and evaluate ZK-ProVer. On supported SV-COMP benchmarks, Phase I validates program-to-CNF translations of bounded verification conditions with reusable verifier checks; against ZKUNSAT [20] on ten UNSAT instances, Phase II achieves an 11.1× geometric-mean verifier speedup and a 410.4× geometric-mean reduction in verifier-side communication.

Paper Organization. The remainder of this paper is organized as follows: Section 2 reviews the technical background. Section 3 presents the two-phase proof relations and AIR constraints. Section 4 describes the implementation and empirical evaluation. Section 5 discusses related work. Section 6 discusses implications and limitations, and Section 7 concludes the paper.

2 Preliminaries

2.1 Zero-Knowledge Proofs and Algebraic Intermediate Representations

Zero-knowledge proofs allow a prover to certify an existential statement without revealing the private witness. Let R be the relation to be proved, with public input u and private witness w . For a true instance $(u, w) \in R$, the prover produces a proof that convinces the verifier of the existence of such a witness, while the verifier learns only the validity of the statement and the public values derived from u . Modern succinct ZKPs realize this goal through *arithmetization*: the relation R is represented by algebraic constraints over a finite field $\mathbb{F}$, the witness is encoded by polynomials over that field, and the verifier checks that the induced polynomial identities hold.

We focus on a trace-based form of arithmetization, in which the witness is organized into one or more row-indexed tables rather than an unstructured collection of gates. A *logical trace table* is represented as a matrix $\mathcal{T} \in \mathbb{F}^{N_T \times M_T}$, where each row corresponds to one step or one logical record, and each column stores one component of the witness or of an auxiliary check. Let $\omega \in \mathbb{F}$ be an element of multiplicative order N_T , i.e., $\omega^{N_T} = \omega^0 = 1$ and $\omega^0, \dots, \omega^{N_T-1}$ are distinct. When $N_T = 2^k$, this condition becomes $\omega^{2^k} = \omega^0$. The row domain is the multiplicative subgroup $H_T = \{\omega^0, \dots, \omega^{N_T-1}\}$. For each column c , the map $\omega^i \mapsto \mathcal{T}[i, c]$ on H_T has a unique interpolant $f_c \in \mathbb{F}[X]$ with $\deg(f_c) < N_T$:

$$f_c(\omega^i) = \mathcal{T}[i, c] \quad 0 \leq i < N_T.$$

Evaluations on H_T therefore recover the trace rows, and multiplication by ω induces the cyclic next-row shift; in particular, $f_c(\omega X)$ denotes the column c shifted forward by one row over H_T .

An *algebraic intermediate representation* (AIR) is the constraint object used to describe such a trace-based computation. We model an AIR as a tuple

$$\text{AIR} = (\{\mathcal{T}_t\}_{t=1}^r, \{\text{Cols}_t\}_{t=1}^r, C_{\text{loc}}, C_{\text{ms}}).$$

Here each trace table $\mathcal{T}_t \in \mathbb{F}^{N_t \times M_t}$ has a schema $\text{Cols}_t = (c_{t,1}, \dots, c_{t,M_t})$ of named columns, C_{loc} is a finite set of local constraint templates, and C_{ms} is a finite set of global multiset interaction templates. Public parameters may appear as constants in these templates. We use *trace* to mean the concrete collection of trace tables $\{\mathcal{T}_t\}_{t=1}^r$, whose rows record the step-by-step witness values for the computation being checked. A trace satisfies the AIR if every local template vanishes on every row of its attached table, and every global interaction induces equal send and receive multisets.

Algebraic intermediate representations use two primitive forms of constraints. For a trace table $\mathcal{T}_t$, let $\mathcal{E}_t(i)$ denote the set of row expressions generated from public constants and terms $\mathcal{T}_t[(i + \delta) \bmod N_t, c]$, where $c \in \text{Cols}_t$ is a named

column and δ is a bounded row offset, using field operations. When the table is clear from context, we write $c[i]$ for the value of column c on row i .

Local algebraic constraints. A local constraint template is a polynomial identity evaluated row-wise. We write

$$C_{\text{loc}} = \bigcup_{t=1}^r C_{\text{loc}}^{(t)}, \quad C_{\text{loc}}^{(t)} = \{P_{t,q}\}_{q=1}^{m_t},$$

where m_t is the number of local templates attached to $\mathcal{T}_t$, and $P_{t,q}$ is a polynomial over $\mathcal{E}_t(i)$. A trace satisfies the q -th local template of table $\mathcal{T}_t$ iff

$$P_{t,q}(\mathcal{T}_t[(i + \delta_1) \bmod N_t, c_1], \dots, \mathcal{T}_t[(i + \delta_k) \bmod N_t, c_k]) = 0 \quad 0 \leq i < N_t.$$

Here $c_1, \dots, c_k$ are columns and $\delta_1, \dots, \delta_k$ are bounded row offsets, such as the current row or the next row.

Global multiset constraints. Many trace-oriented proof systems also need to relate values that are not adjacent in the trace. They do so through weighted multiset equalities. We write $\uplus$ for multiset union with multiplicities, $\{a\}$ for the singleton multiset containing a when it appears in a multiset expression, and $\alpha \cdot M$ for the multiset obtained by multiplying every multiplicity in M by α . We write

$$C_{\text{ms}} = \{(\text{Send}_s, \text{Recv}_s)\}_{s \in S},$$

where each interaction s specifies a send multiset Send_s and a receive multiset Recv_s , each described by a finite collection of tuple generators. A generator has the form

$$g = (t_g, \Phi_g, m_g), \quad \Phi_g = (\phi_{g,1}, \dots, \phi_{g,k}),$$

where t_g identifies the attached trace table, $\phi_{g,1}, \dots, \phi_{g,k} \in \mathcal{E}_{t_g}(i)$ are tuple-field expressions, and $m_g \in \mathcal{E}_{t_g}(i)$ is a multiplicity expression. If g belongs to the description of Send_s or Recv_s , then for every row $0 \leq i < N_{t_g}$ it inserts into that multiset the tuple

$$\Phi_g(i) = (\phi_{g,1}(i), \dots, \phi_{g,k}(i)) \in \mathbb{F}^k$$

with multiplicity $m_g(i)$; a zero multiplicity disables that row. A single global interaction may aggregate generators from multiple trace tables and different row lengths. The interaction s holds iff, for every tuple value $a \in \mathbb{F}^k$, the total multiplicity contributed by all send generators equals the total multiplicity contributed by all receive generators:

$$\sum_{g \in \text{Send}_s} \sum_{i: \Phi_g(i)=a} m_g(i) = \sum_{g \in \text{Recv}_s} \sum_{j: \Phi_g(j)=a} m_g(j).$$

Although we describe the condition above as a multiset equality, the STARK backend enforces it through a LogUp-style argument. Logarithmic derivatives reduce equality of multiplicity-preserving multisets to rational identity checks, which are then compiled into polynomial identities over the evaluation domain [13, 23] and verified using the backend's standard cryptographic machinery. This implementation detail is independent of the AIR-level abstraction in this section: the AIR specifies the required multiset equalities, while the backend provides the mechanism for enforcing them.

Example. Figure 1 illustrates how the two constraint forms can be combined. In the tuple notation above, the AIR has tables $\{\mathcal{T}_{\text{comp}}, \mathcal{T}_{\text{use}}\}$ with schemas

$$\text{Cols}_{\text{comp}} = (x, y, z), \quad \text{Cols}_{\text{use}} = (z', r).$$

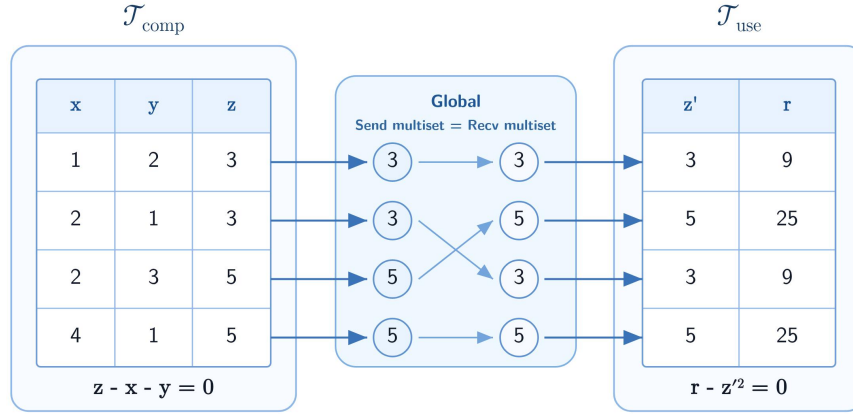


Fig. 1. Example AIR combining row-local arithmetic with a global multiset interaction. The compute table $\mathcal{T}_{\text{comp}}$ locally enforces $z = x + y$ and sends the resulting z -values; the use table $\mathcal{T}_{\text{use}}$ locally enforces $r = (z')^2$ and receives the same multiset of values, possibly in a different row order.

The example computation first produces sums $z = x + y$, then reuses exactly those produced values, possibly in a different order, to compute squares. In the concrete instance shown in the figure, the compute table produces the multiset $\{3, 3, 5, 5\}$, while the use table consumes the same multiset in the order $\{3, 5, 3, 5\}$. The local constraint set C_{loc} contains the two row-wise templates

$$z[i] - x[i] - y[i] = 0, \quad r[j] - (z'[j])^2 = 0,$$

for all rows of the two tables. These constraints establish the arithmetic meaning of each row but do not relate the two tables. The global multiset set C_{ms} contains one interaction whose send generator reads column z from $\mathcal{T}_{\text{comp}}$ and whose receive generator reads column z' from $\mathcal{T}_{\text{use}}$, both with unit multiplicity:

$$\text{Send} = \{(z[i]) : 0 \leq i < N_{\text{comp}}\} = \text{Recv} = \{(z'[j]) : 0 \leq j < N_{\text{use}}\},$$

so the use table can reorder the produced values, but cannot introduce values that were not produced or omit values that were. Together, C_{loc} and C_{ms} form an arithmetization of the complete producer-consumer computation.

An algebraic witness satisfies the arithmetization when all local constraints and all global multiset constraints hold. A STARK prover [2] takes an AIR and a satisfying trace, and produces a succinct argument that the trace satisfies the AIR. At this level of abstraction, the relevant interface is the relation between the AIR, the trace, and the resulting succinct argument. The details of low-degree testing protocols and hash functions are backend-specific.

A zero-knowledge virtual machine (zkVM) replaces task-specific polynomial constraints with a fixed arithmetization of a general machine model [8]. It fixes an instruction set, a machine state, and an algebraic encoding of the machine transition relation. To prove a program run, the prover compiles the computation to a guest program or bytecode for that instruction set, executes it inside the virtual machine, uses the resulting execution trace as the witness, and produces a zero-knowledge proof that each step follows the prescribed machine semantics and yields the claimed public output, without revealing private inputs or intermediate states. Under this abstraction, a zkVM provides a general-purpose

execution-proving interface for computations with complex control flow or irregular semantics. Highly structured checks can instead be expressed directly as specialized AIRs.

2.2 SAT-Based Program Verification

Let P be a program and let $\mathcal{A}$ be a set of assertions or safety properties to be checked. Bounded SAT-based verification fixes a finite execution scope, such as a loop-unrolling bound or a bounded symbolic execution depth, and constructs a formula whose satisfying assignments correspond to executions that violate at least one assertion in $\mathcal{A}$. Thus, the verification problem is reduced to deciding the satisfiability of a Boolean formula.

At a semantic level, let $s_0, \dots, s_k$ denote program states within the bounded scope, $I(s_0)$ the initial-state condition, and $\tau(s_i, s_{i+1})$ the transition relation induced by the program. Let $\text{Bad}_{\mathcal{A}}(s_i)$ denote that some assertion in $\mathcal{A}$ is violated at state s_i . The bounded verification condition can be written abstractly as

$$VC(P, \mathcal{A}, k) = I(s_0) \wedge \bigwedge_{i=0}^{k-1} \tau(s_i, s_{i+1}) \wedge \bigvee_{i=0}^k \text{Bad}_{\mathcal{A}}(s_i).$$

The condition is satisfiable exactly when there exists a concrete bounded execution that triggers an assertion violation. If the condition is unsatisfiable, then no assertion violation exists within the chosen scope.

Standard bounded-model-checking frontends translate this condition into a bit-precise Boolean formula using program transformations such as SSA conversion, bit-vector encoding, path-condition encoding, and bit-blasting into CNF [5, 7]. The resulting CNF formula F is interpreted as a Boolean encoding of the bounded violation condition for P and $\mathcal{A}$.

2.3 SAT Assignments and UNSAT Resolution Proofs

Let $F = \{C_0, \dots, C_{n-1}\}$ be a CNF formula over Boolean variables $X = \{x_0, \dots, x_{m-1}\}$, where each clause C_i is a disjunction of literals and each literal is either x_j or $\neg x_j$. A SAT solver returns one of two results.

SAT assignments. If F is satisfiable, the solver returns an assignment $\alpha : X \rightarrow \{0, 1\}$ such that every clause contains at least one literal evaluating to true under α . In the program-verification setting above, such an assignment encodes a concrete counterexample, including the input values and path choices that trigger an assertion violation.

UNSAT resolution proofs. If F is unsatisfiable, the solver can return a resolution proof [31, 32]. A resolution proof is a sequence of derived clauses $D = \langle D_0, \dots, D_{\ell-1} \rangle$. Let the clause database $E_0, \dots, E_{n+\ell-1}$ contain the original clauses first and the derived clauses afterward:

$$E_i = C_i \quad (0 \leq i < n), \quad E_{n+i} = D_i \quad (0 \leq i < \ell).$$

For a variable p , the binary resolution rule derives

$$\text{Res}_p(A \vee x_p, B \vee \neg x_p) = A \vee B,$$

where A and B contain the non-pivot literals of the two parents. The proof is valid if, for every derived clause D_i , there exist parent identifiers $0 \leq a[i], b[i] < n + i$ and a pivot $p[i]$ such that

$$E_{n+i} = \text{Res}_{p[i]}(E_{a[i]}, E_{b[i]}),$$

and the final derived clause is the empty clause, $D_{\ell-1} = \perp$. The empty clause certifies that no satisfying assignment exists.

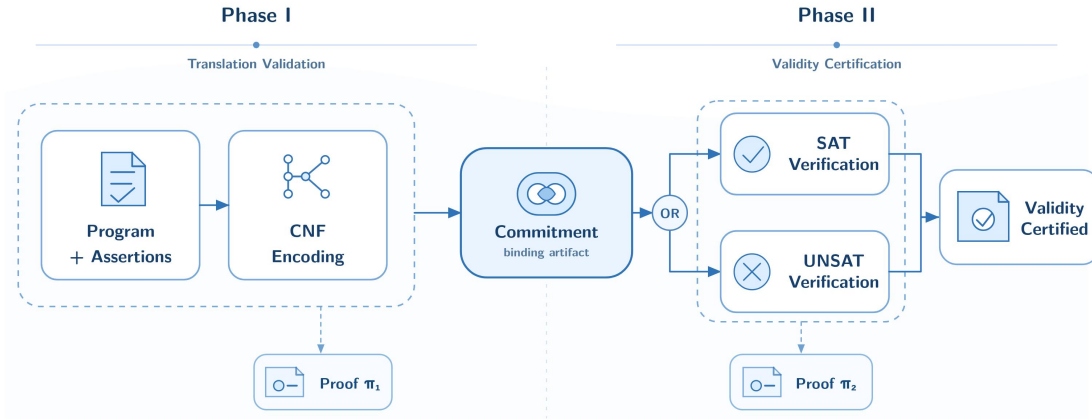


Fig. 2. Overview of the two-phase ZK-ProVer certification framework. Phase I binds the frontend-produced CNF formula to a commitment, and Phase II certifies the solver result for that committed formula through either the SAT or UNSAT branch.

Assignments and resolution proofs create different certification burdens. A SAT assignment is compact but directly exposes the counterexample. In contrast, an UNSAT result is certified by a resolution proof whose size and propagation-based checking cost can be substantial [26, 31]; large SAT-derived refutations in standard clausal proof formats have even reached nearly 200 terabytes [15]. Zero-knowledge certification can therefore serve two complementary goals: hiding the satisfying assignment in the SAT case, and replacing transmission of a large resolution proof in the UNSAT case with a compact proof that convinces the verifier that the CNF formula is unsatisfiable.

3 Zero-Knowledge Certification Framework

Figure 2 gives the high-level structure of ZK-ProVer. Phase I validates the deterministic translation from the source program and assertions to a CNF formula, then publishes a commitment that binds all later proofs to that formula. Phase II consumes the same commitment and certifies exactly one solver outcome: the SAT branch proves the existence of a satisfying assignment, while the UNSAT branch proves the validity of a resolution refutation. The output is a compact certified result rather than the raw counterexample or full UNSAT certificate.

This section specifies the proof relations behind this flow using the constraint vocabulary introduced in Section 2. The Phase I relation is discharged inside a zkVM, whereas the Phase II branches are realized by specialized AIR constraint systems over a Plonky3-based STARK backend. For the Phase II branches, we give the logical table schemas and the local and global multiset constraints that instantiate the AIR tuple from Section 2.1; backend parameters and low-level optimizations are orthogonal to these proof relations.

3.1 Phase I Translation-Validation Relation

Phase I binds the solver input to the source-level verification task. It does not establish the verification outcome itself; rather, it certifies that the solver input is the canonical CNF encoding whose satisfiability or unsatisfiability is subsequently certified in Phase II. Concretely, the zkVM reruns the deterministic frontend that turns a C program with assertions into a CNF formula and checks that the formula used by the solver is exactly the one produced by this

Algorithm 1 Phase I translation validation from C to CNF

Require: C source P and candidate CNF formula F
Ensure: Formula commitment com_F or reject

- 1: $P_0 \leftarrow \text{Normalize}(P)$
- 2: $Q \leftarrow \text{LowerToCFG}(P_0)$
- 3: **if** $Q = \perp$ or Q uses unsupported or cyclic control flow **then**
- 4: reject
- 5: **end if**
- 6: Initialize a CNF builder, an input-symbol cache, and the entry symbolic state
- 7: Compute live variables and process blocks of Q in topological order
- 8: **for** each reachable block b **do**
- 9: Merge incoming guarded states for the live variables of b
- 10: **for** action a in b **do**
- 11: **if** a is an assignment **then**
- 12: Bit-blast the right-hand side and update the symbolic state
- 13: **else if** a is an assertion **then**
- 14: Bit-blast the predicate and record the guarded failing branch
- 15: **end if**
- 16: **end for**
- 17: Propagate the state along goto/branch/return terminators under guards
- 18: **end for**
- 19: $F' \leftarrow$ generated clauses plus a unit clause requiring some recorded failure
- 20: reject unless $F' = F$
- 21:
- 22: **return** $\text{Commit}(F)$

frontend. The accepted relation is

$$R_{\text{enc}}(P, F, \text{com}_F) = 1,$$

where P is the C source processed by deterministic normalization, F is the candidate CNF formula, and $\text{com}_F = \text{Commit}(F)$. Once this relation is proved, the SAT and UNSAT branches can refer only to com_F , rather than exposing the formula or requiring each verifier to rerun the frontend.

Our implementation follows the standard bounded-model-checking pipeline reviewed in Section 2.2. The frontend parses and normalizes the C input, lowers it to a conventional control-flow graph, and then translates that graph to CNF using fixed-width bit-vector symbolic execution and bit-blasting [5, 7]. Variables have fixed scalar widths, each basic block contains assignments and assertions, and the terminator is either a branch, a goto, or a return. Error calls such as `reach_error` are represented as jumps to a distinguished error block. The current implementation accepts acyclic control flow; bounded loops are therefore unrolled before this phase, and cyclic control flow is rejected rather than approximated.

Algorithm 1 specifies the Phase I translation-validation procedure. The symbolic state carried through the block graph consists of a path guard and the current bit-vector value of each live program variable. At control-flow joins, incoming states are merged with guard-selected if-then-else terms. These guarded merges encode the loop-free bounded verification condition whose CNF representation is checked by the zkVM through the concrete encoder.

Expression encoding uses standard bit-vector circuits. Constants become literal vectors, nondeterministic inputs allocate fresh literal vectors, casts resize or sign-extend bit-vectors, Boolean connectives use Boolean gates, and arithmetic, shift, and comparison operators are expanded to bit-level circuits before being converted to CNF by the

builder's equisatisfiable gate encodings. Assertions are handled by recording the guarded condition under which the assertion fails; branches split the current state under the encoded branch predicate; and an explicit error target contributes its reachability guard. At the end, the encoder asserts that at least one recorded failure condition holds, so the CNF is satisfiable exactly when the bounded program has an assertion-violating execution in the supported fragment.

The deterministic ordering of blocks, variables, fresh literals, and clauses makes the regenerated formula byte-for-byte comparable with the supplied formula. If the formulas differ, Phase I rejects. Otherwise, it computes $\text{com}_F = \text{Commit}(F)$ from the canonical CNF representation. Binding later SAT or UNSAT proofs to this commitment prevents a prover from validating one frontend translation and proving a solver result for another formula. The verifier receives com_F and compact metadata, while the CNF clauses and frontend artifacts remain private.

3.2 Shared Phase II Formula Commitment

Both Phase II branches use the same public handle for the private CNF formula. We instantiate $\text{Commit}(F)$ as a Merkle-style vector commitment over a canonical clause encoding [21], using a ZK-friendly sponge hash H such as Poseidon [12]. Let the public metadata be (m, n, w) , where m is the number of variables, n is the number of clauses, and w is the clause-width bound. Each clause C_i is encoded as

$$\text{enc}(C_i) = (i, r_i, \ell_{i,0}, \dots, \ell_{i,w-1}) \in \mathbb{F}^{w+2},$$

where r_i is the number of non-padding literals. A positive literal x_j is encoded as $j + 1$, a negative literal $\neg x_j$ as $-(j + 1)$, and unused slots are padded with 0. The clause order and the literal-slot order are fixed by the deterministic frontend, so this encoding is canonical for the formula produced in Phase I.

The leaf digest is

$$h_i^{(0)} = H(\text{tag}_{\text{leaf}}, \text{enc}(C_i)), \quad 0 \leq i < n.$$

The leaves are padded to the next power of two with a fixed empty digest. The internal nodes are computed by

$$h_i^{(r+1)} = H(\text{tag}_{\text{node}}, h_{2i}^{(r)}, h_{2i+1}^{(r)}),$$

and the root is the public formula commitment com_F . Under the collision resistance of H , this root binds the prover to one ordered CNF formula vector with the declared metadata.

The SAT and UNSAT AIRs below include this same commitment subtrace: their private formula columns are first checked against the canonical clause encoding, and the hash subtrace recomputes the root com_F . The branch-specific constraints then consume the same committed formula, either by checking a satisfying assignment or by checking a resolution refutation. Thus the verifier receives only com_F , compact metadata, and the STARK proof, while the formula clauses and branch-specific witnesses remain private.

3.3 SAT Branch: Assignment AIR

The SAT branch proves that the committed formula has a satisfying assignment. We instantiate the AIR tuple from Section 2.1 as

$$\text{AIR}_{\text{sat}} = (\{\mathcal{T}_{\text{occ}}, \mathcal{T}_{\text{sort}}\}, \{\text{Cols}_{\text{occ}}, \text{Cols}_{\text{sort}}\}, C_{\text{loc}}^{\text{sat}}, C_{\text{ms}}^{\text{sat}}).$$

All clause and variable identifiers are 0-based: clauses are indexed by $\{0, \dots, n-1\}$ and variables by $\{0, \dots, m-1\}$. The occurrence table has one row per literal occurrence, with schema

$$\text{Cols}_{\text{occ}} = (\text{clause}, \text{pos}, \text{var}, \text{sign}, \text{val}, \text{truth}, \text{acc}, \text{isFirst}, \text{isLast}, \mathbf{aux}_{\text{occ}}).$$

Here var is the variable identifier, $\text{sign} \in \{0, 1\}$ selects the positive or negative literal, val is the private assignment bit for that occurrence, truth is the induced literal truth value, and acc is the clause-local prefix accumulator. The sorted table contains the same assignment pairs ordered by variable identifier:

$$\text{Cols}_{\text{sort}} = (\text{var}^s, \text{val}^s, \text{sameNext}, \mathbf{aux}_{\text{sort}}).$$

Here and below, a bold $\mathbf{aux}$ entry denotes a finite family of auxiliary columns rather than a single column. These auxiliary columns contain bounded-limb witnesses for range checks and selector witnesses used by the local templates. For example, a variable identifier j is constrained to satisfy $0 \leq j < m$ by range-checking the small integer $m-1-j$, and a clause identifier c is constrained similarly by range-checking $n-1-c$.

To certify satisfiability, the trace must establish three properties.

Literal truth. Every occurrence row must compute the truth value of its literal under the private assignment bit stored in that row. For a literal occurrence (j, s) , where $j \in \{0, \dots, m-1\}$, $s = 0$ denotes x_j , and $s = 1$ denotes $\neg x_j$, the occurrence table uses a local algebraic constraint

$$t = v + s - 2sv.$$

Together with Booleanity constraints for s , v , and t , this row-wise identity enforces $t = v$ for positive literals and $t = 1 - v$ for negative literals. This property belongs entirely to $C_{\text{loc}}^{(\text{occ})}$.

Clause satisfaction. For each clause, the occurrence rows maintain a prefix-OR accumulator over the truth values in that clause. If a clause has literal truth values $(t_1, \dots, t_r)$, the accumulator satisfies

$$o_1 = t_1, \quad o_{q+1} = o_q + t_{q+1} - o_q t_{q+1}.$$

Local selector columns mark the first and last occurrence of each clause, so the same row-wise polynomial templates apply to every row while enforcing the appropriate boundary cases. At the last occurrence of the clause, the AIR requires $o_r = 1$. Hence the local constraints on $\mathcal{T}_{\text{occ}}$ prove that every clause contains at least one literal whose truth value is 1.

Assignment coherence. The literal and clause constraints above are not sufficient by themselves, because two occurrences of the same variable could otherwise carry different private values. The AIR enforces coherence by combining one global multiset constraint with local constraints on the sorted table. The assignment-coherence interaction in $C_{\text{ms}}^{\text{sat}}$ is

$$\text{Send}_{\text{asg}} = \{(\text{var}[i], \text{val}[i]) : 0 \leq i < N_{\text{occ}}\} = \text{Recv}_{\text{asg}} = \{(\text{var}^s[j], \text{val}^s[j]) : 0 \leq j < N_{\text{sort}}\}.$$

Both sides are generated row-wise from tuple-field expressions over their attached tables. The sorted table then uses local constraints to enforce nondecreasing variable identifiers and adjacency coherence,

$$\text{var}^s[i] = \text{var}^s[i+1] \Rightarrow \text{val}^s[i] = \text{val}^s[i+1].$$

Together, the global multiset equality and the sorted-side local constraints ensure that all occurrences of the same variable use a single assignment bit.

These three properties define the SAT AIR. Formally, $C_{\text{loc}}^{\text{sat}}$ contains the Booleanity, literal-truth, prefix-OR, boundary, sorting, and adjacency templates described above, while $C_{\text{ms}}^{\text{sat}}$ contains the occurrence-to-sorted multiset equality. If a trace satisfies this AIR, define $\alpha(x_j)$ to be the value attached to any occurrence of x_j . Assignment coherence makes this definition well formed. Literal truth then shows that each t is the truth value of the corresponding literal under α , and clause satisfaction shows that every clause has a true literal. Thus $\alpha \models F$. Conversely, any satisfying assignment can fill the occurrence table, prefix-OR columns, and sorted table so that all local and global multiset constraints hold.

THEOREM 3.1. *Assume the Phase I commitment com_F is binding. If the STARK verifier accepts the SAT AIR, then the private trace encodes a satisfying assignment for the formula F bound by com_F . Conversely, any valid satisfying assignment for F can be encoded as an accepting SAT trace.*

Proof. Completeness follows by filling $\mathcal{T}_{\text{occ}}$ with the formula literal occurrences, the assignment values, and the induced prefix-OR values, and by filling $\mathcal{T}_{\text{sort}}$ with the same (j, v) pairs sorted by identifier. All local templates and the global multiset equality then hold. For soundness, any accepting trace satisfies literal truth, clause satisfaction, and assignment coherence as argued above, so it defines a global assignment $\alpha \models F$. Zero knowledge and computational soundness are inherited from the underlying STARK backend; the verifier sees only the public commitment, compact metadata, and proof, not the formula clauses or assignment values.

3.4 UNSAT Branch: Resolution-Proof AIR

The UNSAT branch proves that the committed formula has a resolution refutation. Let the input CNF formula be $F = \{C_0, \dots, C_{n-1}\}$, and let the private resolution witness contain derived clauses

$$D = \langle D_0, \dots, D_{\ell-1} \rangle.$$

For each D_i , the witness records a left parent identifier $L[i]$, a right parent identifier $R[i]$, and a pivot variable $p[i]$. Logically, as in a conventional resolution verifier, the clauses are addressed through one ordered database

$$E_j = \begin{cases} C_j, & 0 \leq j < n, \\ D_{j-n}, & n \leq j < n + \ell, \end{cases}$$

so that D_i has database identifier $n + i$. The AIR does not store this logical database as a single uniform table. Instead, it separates input CNF clauses from derived resolution clauses.

We instantiate the AIR tuple as

$$\text{AIR}_{\text{unsat}} = (\{\mathcal{T}_{\text{init}}, \mathcal{T}_{\text{der}}\}, \{\text{Cols}_{\text{init}}, \text{Cols}_{\text{der}}\}, C_{\text{loc}}^{\text{unsat}}, C_{\text{ms}}^{\text{unsat}}).$$

The input-clause table has one row for each input CNF clause and contains only the columns needed for formula entries:

$$\text{Cols}_{\text{init}} = (\text{id}^{\text{init}}, \text{use}^{\text{init}}, \text{clause}_0^{\text{init}}, \dots, \text{clause}_{w-1}^{\text{init}}, \text{aux}_{\text{init}}).$$

The derived table has one row for each resolution step, with schema

$$\begin{aligned} \text{Cols}_{\text{der}} = & (\text{id}^{\text{der}}, L, R, p, \text{use}^{\text{der}}, \\ & \text{clause}_0^{\text{der}}, \dots, \text{clause}_{w-1}^{\text{der}}, \\ & \text{left}_0, \dots, \text{left}_{w-1}, \text{right}_0, \dots, \text{right}_{w-1}, \text{aux}_{\text{der}}). \end{aligned}$$

The split avoids assigning irrelevant values to the columns that are meaningful only for derived resolution steps on the n input-clause rows. The columns $\text{clause}_k^{\text{init}}$ and $\text{clause}_k^{\text{der}}$ store input and derived clauses, respectively; left_k and right_k store copied parent clauses for derived rows. The columns use^{init} and use^{der} store parent-use multiplicities. The auxiliary column families $\mathbf{aux}_{\text{init}}$ and $\mathbf{aux}_{\text{der}}$ contain bounded-limb witnesses, inverse witnesses for nonzero checks, padding flags, pivot-position flags, keep flags, and fingerprint products. Identifiers are 0-based for clauses. The identifier columns are deterministic public columns: their polynomial commitments are exposed to the verifier, so the prover and verifier share the same row identifiers. In particular, $\text{id}^{\text{init}} = 0, \dots, n - 1$ fixes the number of input CNF clauses, and $\text{id}^{\text{der}} = n, \dots, n + \ell - 1$ fixes the number of derived clauses. Literal column value 0 is reserved for padding. Pivot values use the nonzero variable identifiers of the literal encoding: $1 \leq p[i] \leq m$, and the corresponding positive and negative literals are $+p[i]$ and $-p[i]$. Here $-p[i]$ denotes the additive inverse of $p[i]$ in the field; under the canonical integer representation of a prime field $\mathbb{F}_q$, it appears as the wraparound value $q - p[i]$. For a derived-row index i , we write $L[i]$, $R[i]$, and $p[i]$ for the values of columns L , R , and p on row i of $\mathcal{T}_{\text{der}}$.

The UNSAT AIR is a trace-based encoding of four properties that are necessary and sufficient for this database to be a valid refutation.

Database well-formedness. The two tables must form the ordered clause database $E_0, \dots, E_{n+\ell-1}$. The public identifier columns already fix the input-clause table as entries $E_0, \dots, E_{n-1}$ and the derived table as entries $E_n, \dots, E_{n+\ell-1}$; in particular, $\text{id}^{\text{der}}[i] = n + i$. Input validation requires $n + \ell < 2^{24}$, so every public database identifier lies in the bounded integer range $[0, 2^{24})$. Clause columns shorter than the width bound w are padded with zeros.

The parent identifiers $L[i]$ and $R[i]$ are not range-checked as standalone identifier columns. Instead, the parent-clause binding interaction below forces each pair $(L[i], P^L[i])$ and $(R[i], P^R[i])$ to match a database entry whose first component is one of the public identifiers. Thus $L[i]$ and $R[i]$ inherit the same bounded identifier range from the public id columns. Here a range check for a value $x < 2^{8s}$ uses auxiliary limbs $x_0, \dots, x_{s-1}$, each constrained to lie in $[0, 2^8)$, together with the reconstruction constraint $x = \sum_{h=0}^{s-1} 2^{8h} x_h$ over the base field. Identifier-related quantities use $s = 3$ limbs, corresponding to the bound 2^{24} . For each derived row i , local constraints then enforce that the referenced parents are earlier than the current derived clause by range-checking the nonnegative gaps

$$(n + i) - 1 - L[i], \quad (n + i) - 1 - R[i],$$

as 2^{24} -bounded integers. For example, checking $(n + i) - 1 - L[i] < 2^{24}$ rules out field wraparound for this difference; since both $n + i$ and $L[i]$ are already 2^{24} -bounded identifiers, the field equality then represents the ordinary integer inequality $L[i] < n + i$. The same reasoning applies to $R[i]$. The pivot bound $1 \leq p[i] \leq m$ is enforced analogously by applying the same limb-based range check to $p[i] - 1$ and $m - p[i]$, where m is the public variable bound. Combined with parent-clause binding, these checks imply $0 \leq L[i], R[i] < n + i$. These templates are elements of $C_{\text{loc}}^{(\text{der})}$.

Parent-clause binding. A derived row must use the actual earlier database entries named by $L[i]$ and $R[i]$, not arbitrary copied clauses. This nonlocal condition is expressed as a global multiset constraint. Let

$$C^{\text{cols}}[r] = (\text{clause}_0^{\text{init}}[r], \dots, \text{clause}_{w-1}^{\text{init}}[r])$$

denote the input clause stored on row r of $\mathcal{T}_{\text{init}}$. For derived row i , let

$$D^{\text{cols}}[i] = (\text{clause}_0^{\text{der}}[i], \dots, \text{clause}_{w-1}^{\text{der}}[i])$$

denote the derived clause, and let

$$P^L[i] = (\text{left}_0[i], \dots, \text{left}_{w-1}[i]), \quad P^R[i] = (\text{right}_0[i], \dots, \text{right}_{w-1}[i])$$

be the copied left and right parent clauses. The send side inserts $(\text{id}^{\text{init}}[r], C^{\text{cols}}[r])$ from each input row and $(\text{id}^{\text{der}}[i], D^{\text{cols}}[i])$ from each derived row, with their respective use multiplicities. The receive side inserts $(L[i], P^L[i])$ and $(R[i], P^R[i])$ from each derived row. The parent-binding interaction in $C_{\text{ms}}^{\text{unsat}}$ is

$$\begin{aligned} \text{Send}_{\text{par}} &= \bigoplus_{r=0}^{n-1} \text{use}^{\text{init}}[r] \cdot \{(\text{id}^{\text{init}}[r], C^{\text{cols}}[r])\} \\ &\quad \uplus \bigoplus_{i=0}^{\ell-1} \text{use}^{\text{der}}[i] \cdot \{(\text{id}^{\text{der}}[i], D^{\text{cols}}[i])\}, \\ \text{Recv}_{\text{par}} &= \bigoplus_{i=0}^{\ell-1} (\{(L[i], P^L[i])\} \uplus \{(R[i], P^R[i])\}), \\ \text{Send}_{\text{par}} &= \text{Recv}_{\text{par}}. \end{aligned}$$

The use columns therefore equal the number of references to the corresponding database identifiers: $\text{use}^{\text{init}}[r]$ counts references to input identifier r , and $\text{use}^{\text{der}}[i]$ counts references to derived identifier $n + i$. Combined with database well-formedness, this prevents a prover from using a fabricated parent clause or a future derived clause.

Resolution-step validity. Once the parent clauses are bound, each derived row must locally enforce one resolution inference. The copied parents $P^L[i]$ and $P^R[i]$ must contain opposite pivot literals: $+p[i]$ in the left parent and $-p[i]$ in the right parent. This requirement is enforced by the following local constraints. For each literal slot k , define the row expressions

$$d_k^L[i] = \text{left}_k[i] - p[i], \quad d_k^R[i] = \text{right}_k[i] + p[i].$$

The derived table contains Boolean flags $\pi_k^L[i]$, $\pi_k^R[i]$ and inverse witnesses $\eta_k^L[i]$, $\eta_k^R[i]$. The local templates enforce

$$\begin{aligned} \pi_k^L[i](\pi_k^L[i] - 1) &= 0, & \pi_k^L[i] d_k^L[i] &= 0, & (1 - \pi_k^L[i])(\eta_k^L[i] d_k^L[i] - 1) &= 0, \\ \pi_k^R[i](\pi_k^R[i] - 1) &= 0, & \pi_k^R[i] d_k^R[i] &= 0, & (1 - \pi_k^R[i])(\eta_k^R[i] d_k^R[i] - 1) &= 0. \end{aligned}$$

Thus $\pi_k^L[i] = 1$ exactly when $\text{left}_k[i] = p[i]$, and $\pi_k^R[i] = 1$ exactly when $\text{right}_k[i] = -p[i]$. The row also enforces

$$\sum_{k=0}^{w-1} \pi_k^L[i] = 1, \quad \sum_{k=0}^{w-1} \pi_k^R[i] = 1.$$

Thus each derived row contains exactly one positive pivot literal in its left parent and exactly one negative pivot literal in its right parent. After these two pivot occurrences are removed, the derived clause must be exactly the disjunction of the remaining literals from the two parents. We check this by matching the left-parent remainder and the right-parent remainder against the literals selected from $D^{\text{cols}}[i]$ through two row-local multiset equalities. Since value 0 denotes a padding literal, the check must ignore literal positions whose stored value is 0. Let $\zeta_k^L[i]$, $\zeta_k^R[i]$, and $\zeta_t^D[i]$ be padding flags, constrained by zero-test templates so that they are 1 exactly when the corresponding literal value is 0: for a literal value x , flag ζ , and inverse witness θ , the templates are

$$\zeta(\zeta - 1) = 0, \quad \zeta x = 0, \quad (1 - \zeta)(\theta x - 1) = 0.$$

Define the source-side activity flags

$$a_k^L[i] = (1 - \zeta_k^L[i])(1 - \pi_k^L[i]), \quad a_k^R[i] = (1 - \zeta_k^R[i])(1 - \pi_k^R[i]).$$

Thus $a_k^L[i]$ selects exactly the non-padding literals of the left parent other than the positive pivot, and $a_k^R[i]$ selects exactly the non-padding literals of the right parent other than the negative pivot.

The derived clause uses Boolean keep flags $\kappa_t^L[i]$ and $\kappa_t^R[i]$. They indicate whether the literal $\text{clause}_t^{\text{der}}[i]$ is accounted for by the left source multiset, the right source multiset, or both. The AIR enforces

$$\kappa_t^\sigma[i](\kappa_t^\sigma[i] - 1) = 0 \quad (\sigma \in \{L, R\}), \quad 1 - \zeta_t^D[i] = \kappa_t^L[i] + \kappa_t^R[i] - \kappa_t^L[i]\kappa_t^R[i],$$

so every non-padding derived literal is selected by at least one side, while zero-valued padding literals are selected by neither side.

It remains to require that the selected literals match the source literals as multisets. For a verifier-sampled fingerprint challenge ρ , define

$$\chi(b, x) = 1 - b + b(\rho + x),$$

so $\chi(0, x) = 1$ and $\chi(1, x) = \rho + x$. The left-side equality is encoded by the constraints

$$\sum_{k=0}^{w-1} a_k^L[i] = \sum_{t=0}^{w-1} \kappa_t^L[i],$$

$$\prod_{k=0}^{w-1} \chi(a_k^L[i], \text{left}_k[i]) = \prod_{t=0}^{w-1} \chi(\kappa_t^L[i], \text{clause}_t^{\text{der}}[i]).$$

¹ The right-side equality is identical, replacing a_k^L , left_k , and κ_t^L by a_k^R , right_k , and κ_t^R . The row therefore enforces

$$P^L[i] = A \vee x_{p[i]}, \quad P^R[i] = B \vee \neg x_{p[i]} \implies D^{\text{cols}}[i] = A \vee B.$$

Terminal contradiction. The final semantic row must be a derived row whose clause contains only padding zeros. This is again a local constraint, using the final-row selector together with zero constraints on all literal columns. It enforces $D_{\ell-1} = \perp$, turning locally valid resolution derivations into a refutation.

These four properties define the UNSAT AIR. The local constraint set $C_{\text{loc}}^{\text{unsat}}$ contains the row-order, range, pivot-removal, row-local multiset, and terminal templates over the initial and derived tables. The global multiset set $C_{\text{ms}}^{\text{unsat}}$ contains the parent-clause binding interaction. If all these constraints hold, then every derived row is a valid resolution consequence of earlier database rows, and the final derived row is the empty clause.

THEOREM 3.2. *Assume the Phase I commitment com_F is binding and the Plonky3 STARK verifier accepts the UNSAT AIR for public metadata (m, n, w) . Then the private witness contains a valid resolution refutation of the formula F bound by com_F . Conversely, any valid resolution refutation of F can be encoded as an accepting UNSAT trace.*

Proof. For completeness, an honest prover writes the input CNF clauses into $\mathcal{T}_{\text{init}}$, and writes the derived clauses, parent identifiers, pivots, copied parent clauses, and auxiliary range and fingerprint witnesses into $\mathcal{T}_{\text{der}}$. The parent-use multiplicities make the global multiset interaction balanced, each row-local resolution check holds, and the last derived row is the empty clause.

¹The notation above states the row-local multiset check compactly. The AIR does not materialize the products as degree- w constraints. Instead, it stores prefix counts and prefix products in auxiliary columns and enforces one constant-degree recurrence per literal position; the final prefix values are then compared.

For soundness, consider any accepting trace. Database well-formedness gives an acyclic clause database $E_0, \dots, E_{n+\ell-1}$ whose first n entries are represented by $\mathcal{T}_{\text{init}}$ and whose remaining entries are represented by $\mathcal{T}_{\text{der}}$. Parent-clause binding guarantees that the copied parents in each derived row are exactly the earlier database entries named by $L[i]$ and $R[i]$. Resolution-step validity then implies that each derived entry E_{n+i} , for $0 \leq i < \ell$, is the resolvent of two earlier entries. By induction on the database identifier, every derived clause is derivable from F by resolution. Terminal contradiction forces the final derived clause to be empty, so the trace contains a resolution refutation of F . Zero knowledge and computational soundness are inherited from the underlying STARK backend; the verifier receives only the public commitment, compact metadata, and STARK proof, rather than the full clause database or resolution witness.

4 Experimental Evaluation

Implementation. ZK-ProVer is implemented in Rust and integrated with a standard SAT-based verification workflow. The implementation is open source, and the repository also includes Coq proof scripts for the SAT and UNSAT AIRs described in Sections 3.3 and 3.4². Phase I uses SP1 [29], a RISC-V zkVM, to execute the translation-validation guest program and prove that the supplied CNF formula matches the deterministic program-to-CNF translation. Phase II implements the SAT and UNSAT AIRs over a Plonky3-based STARK backend [24]. The frontend supports a scalar C subset covering integer and Boolean variables, assignments, arithmetic and bitwise expressions, comparisons, conditional branches, and assertions. It does not support pointers, arrays, heap objects, or native loop/recursion reasoning; bounded-loop benchmarks are therefore source-level unrolled before encoding. For UNSAT experiments, the CNF formula is solved with PicoSAT [4]. We use PicoSAT’s trace output to obtain a solver certificate and unfold it with the proof-processing scripts from the ZKUNSAT artifact [20], producing the resolution witness consumed by our Phase II UNSAT backend.

Benchmark. The evaluation uses two benchmark families. The first family consists of pointer-free scalar C programs supported by the evaluated zk-prover frontend and drawn from the SV-COMP validation-crafted and loop benchmark styles. Because the frontend accepts acyclic control flow in this implementation, bounded-loop programs are expanded before encoding.³ This family includes the official SV-COMP fixtures supported by the frontend, non-degenerate scalar fixtures from the repository, and bounded SV-COMP loop benchmarks after source-level unrolling. These programs exercise the C-to-CFG lowering and CFG-to-CNF encoding path proved by Phase I. Table 1 summarizes the converted CNF instances and solver outcomes included in the evaluation. We exclude bitvector-regression fixtures that collapse to degenerate one-variable CNFs, since they do not provide representative evidence for verification-condition encoding.

The second family is used for the UNSAT baseline comparison in RQ2. It contains ten unfolded resolution proofs obtained from the ZKUNSAT artifact and from SV-COMP-derived formulas generated with the same PicoSAT trace workflow. The instances are selected to span resolution length from 37 to 15,445 steps and clause width from 7 to 96. This family isolates the Phase II UNSAT cost from the frontend and focuses on the dimensions that dominate resolution-checking arithmetization: derivation length and clause width.

Experimental Environment. Experiments are run on Ubuntu 22.04.5 LTS under WSL2 (kernel 6.6.87.2). The workstation is equipped with an AMD Ryzen 9 9950X3D processor with 16 cores, 32 hardware threads, and 64 GiB of RAM. The

²<https://github.com/WindOctober/zk-prover>

³For the bounded-loop rows in Table 1, we manually unroll the fixed-bound loops before passing the programs to the frontend. This preprocessing is only used to obtain acyclic inputs for the implementation and is not counted as native loop support.

Table 1. Converted SV-COMP-style C fixtures used by the evaluated implementation. All rows avoid pointers and arrays; bounded-loop rows are unrolled before frontend encoding. *LoC* is the source-line count; *Steps* and *Blocks* are CFG-encoding statistics; *Vars* and *Clauses* are the DIMACS CNF dimensions.

Fixture	Kind	LoC	Steps	Blocks	Vars	Clauses	Result
Official SV-COMP fixtures							
if.c	control flow	27	8	9	628	2,010	SAT
ternary.c	control flow	22	4	5	254	572	SAT
SV-COMP-style scalar fixtures							
diamond_safe.c	branch arithmetic	26	8	9	968	3,001	UNSAT
range_arith_safe.c	range arithmetic	26	8	9	918	2,820	UNSAT
branch_sum_safe.c	nested branches	36	18	19	1,691	5,585	UNSAT
nested_guard_safe.c	guarded branches	33	16	17	1,189	3,420	UNSAT
range_pack_2_safe.c	packed ranges	37	14	15	1,839	5,650	UNSAT
Bounded SV-COMP loops, source-level unrolled							
down_3_unrolled.c	loop invariant	32	26	27	1,815	5,836	UNSAT
large_const_2_unrolled.c	loop invariant	60	36	37	1,705	5,016	UNSAT
bhmr2007_3_bug_unrolled.c	path invariant	46	23	24	1,099	3,534	SAT

implementation is written in Rust and built with rustc 1.93.0-nightly and cargo 1.93.0-nightly. Unless otherwise stated, benchmark results are reported as medians over repeated executions.

Evaluation. The empirical study is organized around two research questions.

- **RQ1. Effectiveness on supported C benchmarks.** Can ZK-ProVer certify verification results for supported C programs after they are translated into CNF?
- **RQ2. Efficiency of Phase II for UNSAT.** How do prover time, verifier time, communication, and interaction compare with the current state of the art for zero-knowledge UNSAT certification?

Baseline for RQ2. ZKUNSAT [20] is the closest state-of-the-art system for zero-knowledge certification of UNSAT claims. Its protocol hides a resolution refutation for a public formula by encoding clauses as low-degree polynomials, checking weakened-resolution steps through polynomial equality tests, and using zero-knowledge random-access arguments to fetch the private parent clauses. The implementation is built on the EMP-toolkit interactive zero-knowledge stack for circuits and polynomial checks [30], and its artifact provides the proof-processing scripts used in our baseline workflow. We compare against ZKUNSAT only on the UNSAT branch, where both systems certify a resolution-style refutation.

4.1 RQ1: Supported C Benchmarks

For **RQ1**, we evaluate the certification path on the ten supported C fixtures in Table 1. Table 2 reports costs for Phase I SP1 translation validation and for the Phase II Plonky3 AIR proofs.

The results show that the evaluated frontend produces nondegenerate CNF instances for all ten supported programs, while Phase I remains the dominant cost. Local SP1 translation-validation proofs complete on nine fixtures, taking 120.0–595.6 s and producing 2.67–5.67 MiB proofs; the largest constant-heavy fixture exceeds the memory budget. Phase II is substantially lighter on the same instances: SAT assignment proofs finish in 1.32–2.13 s, and committed UNSAT

Table 2. RQ1 proof costs for supported C fixtures. Each phase entry is *prove(s) / verify(ms) / proof(KiB)*. OOM denotes a local SP1 proof run that exceeded the 64 GiB workstation memory budget; the reproduction script records this failure explicitly.

Fixture	Branch	P1: SP1	P2: Plonky3
if.c	SAT	239.6 / 88.2 / 2786.4	1.82 / 340.0 / 71.2
ternary.c	SAT	120.0 / 88.3 / 2739.5	1.32 / 297.0 / 58.4
diamond_safe.c	UNSAT	320.4 / 142.6 / 4235.2	4.42 / 351.5 / 146.1
range_arith_safe.c	UNSAT	312.0 / 133.5 / 4228.8	3.12 / 282.8 / 74.1
branch_sum_safe.c	UNSAT	530.0 / 191.6 / 5758.6	3.60 / 257.4 / 87.2
nested_guard_safe.c	UNSAT	345.2 / 138.8 / 4258.1	3.30 / 297.7 / 93.3
range_pack_2_safe.c	UNSAT	512.8 / 185.5 / 5768.6	2.91 / 328.3 / 80.9
down_3_unrolled.c	UNSAT	595.6 / 184.3 / 5808.5	11.99 / 565.8 / 231.5
large_const_2_unrolled.c	UNSAT	OOM	4.39 / 362.3 / 93.4
bhmr2007_3_bug_unrolled.c	SAT	503.2 / 183.1 / 5751.9	2.13 / 414.2 / 78.1

Table 3. RQ2 comparison between ZK-ProVer and ZKUNSAT on UNSAT resolution proofs. ZK-ProVer communication is the non-interactive proof size; ZKUNSAT communication is the verifier-visible interactive transcript. $V\times$ is ZKUNSAT verifier time divided by ZK-ProVer verifier time, and $C\times$ is ZKUNSAT communication divided by ZK-ProVer proof size.

Instance	Scale		ZK-ProVer			ZKUNSAT		Gain	
	Steps	Width	Prove (s)	Verify (ms)	Proof (KiB)	Verify (s)	Comm. (MiB)	$V\times$	$C\times$
floppy2	37	7	5.80	348.1	241.3	0.34	4.84	0.97x	20.5x
nan	140	53	0.73	58.4	267.9	0.48	10.13	8.17x	38.7x
cdc8	2,083	71	33.11	1,221.0	466.3	5.01	109.59	4.10x	240.7x
eca6	4,238	86	39.23	1,440.0	520.4	12.61	234.16	8.76x	460.8x
qat6	4,971	70	32.92	1,264.0	464.4	13.29	282.28	10.52x	622.5x
eca8	7,388	90	40.17	1,439.0	534.4	21.95	417.30	15.26x	799.6x
eca10	9,132	92	40.72	1,426.0	541.9	28.59	525.77	20.05x	993.6x
eca12	11,943	94	42.00	1,507.0	549.2	37.81	694.50	25.09x	1,294.8x
eca13	13,364	95	43.50	1,573.0	553.4	43.02	780.79	27.35x	1,444.8x
eca14	15,445	96	56.40	1,519.0	556.7	68.20	904.00	44.90x	1,662.9x

proofs finish in 2.91–11.99 s. The largest Phase II cost occurs on `down_3_unrolled.c`, whose resolution witness has 4,525 steps, indicating that UNSAT proving cost is driven more by witness size than by source size alone.

4.2 RQ2: UNSAT Phase II Against ZKUNSAT

For RQ2, we compare the UNSAT branch of ZK-ProVer against ZKUNSAT [20]. The two systems expose different cost profiles. ZKUNSAT avoids producing a reusable non-interactive proof, but each verifier must run an interactive verification protocol and receive a large transcript. ZK-ProVer instead performs STARK proof generation once and produces a compact non-interactive proof that can be checked offline by multiple verifiers. Table 3 reports the single-run comparison on the ten UNSAT instances.

The results quantify the verifier-side benefit of non-interactive certification. Across all ten instances, ZK-ProVer improves verifier time by a geometric mean of 11.1 $\times$ and a median of 12.9 $\times$. On the non-trivial rows with more than 2,000 resolution steps, the geometric-mean verifier speedup rises to 15.6 $\times$. This advantage matters in multi-party settings: a ZK-ProVer proof can be generated once and then checked offline by each verifier, while ZKUNSAT requires each verifier to participate in an interactive verification run. The communication gap is larger. ZK-ProVer proof sizes

stay between 241.3 and 556.7 KiB, whereas ZKUNSAT transcripts range from 4.84 to 904.00 MiB. This corresponds to 20.5 $\times$ to 1,662.9 $\times$ less verifier-side communication, with a geometric mean of 410.4 $\times$.

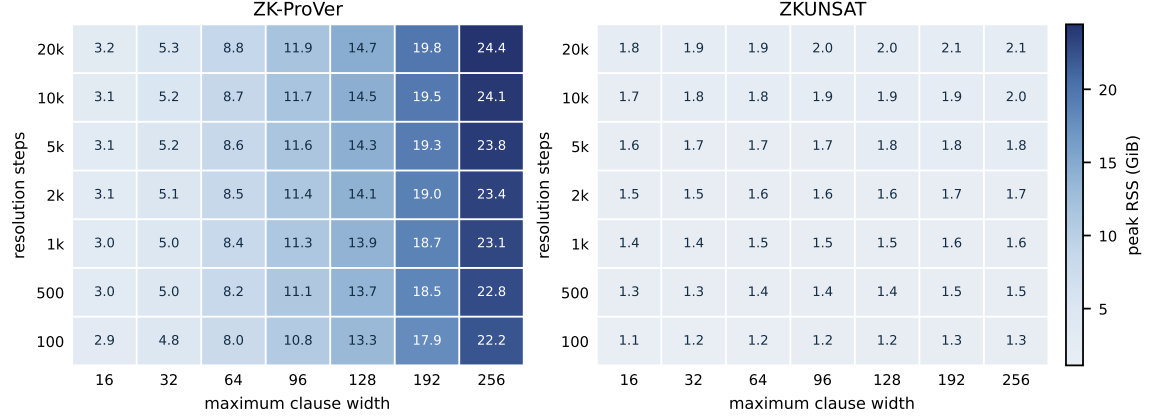


Fig. 3. Measured peak resident memory in a controlled sweep over resolution length and maximum clause width. Each cell reports peak RSS in GiB for the corresponding configuration.

The tradeoff is prover-side memory, especially as the maximum clause width increases. Figure 3 reports a controlled memory sweep over resolution length and width. For ZK-ProVer, increasing the width from 16 to 256 raises peak memory from 2.9–3.2 GiB to 22.2–24.4 GiB across the measured resolution lengths. Increasing the number of resolution steps has a smaller but visible effect at a fixed width; for example, at width 256, peak memory grows from 22.2 GiB at 100 steps to 24.4 GiB at 20k steps. ZKUNSAT remains much flatter over the same grid, ranging from 1.1 to 2.1 GiB. This behavior follows from the current ZK-ProVer trace layout, whose number of columns grows linearly with the maximum clause width; after low-degree extension and commitment construction, the prover keeps several large intermediate matrices. Thus, RQ2 confirms the expected tradeoff: ZK-ProVer substantially reduces verifier work and communication, but wide clauses increase prover memory and motivate a more compact UNSAT AIR layout.

We do not include a SAT-side baseline comparison because there is no directly comparable existing system aimed at the same zkSAT setting. SAT certification checks an assignment against a formula and therefore lacks the resolution-refutation reasoning burden that makes UNSAT certification the more appropriate comparison target.

5 Related Work

ZK for UNSAT. The closest work to our UNSAT branch is ZKUNSAT by Luo et al. [20], which gives an interactive zero-knowledge protocol for proving that a public Boolean formula is unsatisfiable while hiding a private resolution proof. Its main technical contribution is an algebraic encoding of resolution: clauses are represented as low-degree polynomials, resolution is relaxed to a weak-resolution relation, and parent clauses are fetched using zero-knowledge random-access arguments. The implementation is built over the EMP-toolkit interactive zero-knowledge library for Boolean/arithmetic circuits and polynomials [30].

ZK-ProVer is complementary to this line but differs in both target and protocol model. First, ZKUNSAT proves only a solver-level UNSAT claim for a public formula. ZK-ProVer instead targets SAT-based program verification end to end: Phase I validates the frontend-to-CNF translation from the source program and assertions and binds

the resulting CNF formula, and Phase II certifies either the SAT or UNSAT solver result for that same committed formula. Second, our UNSAT branch is non-interactive and produces an offline-verifiable proof. This distinction is not a mechanical application of the Fiat–Shamir heuristic [10]. Fiat–Shamir is normally applied to public-coin interactive proofs or arguments, with security proved in the random-oracle model and with additional care for multi-round protocols [1]. ZKUNSAT, by contrast, is presented in a hybrid model built from interactive commit-and-prove, clause, and random-access functionalities, and its implemented protocol stack is not a single public-coin polynomial IOP to which Fiat–Shamir applies as a drop-in compiler.

The arithmetization choices also differ. ZKUNSAT relies on a stateful zero-knowledge random-access array to retrieve parent clauses during the hidden resolution derivation. Our AIR avoids implementing such a stateful interactive memory abstraction. Instead, the parent clauses named by each derived row are materialized as witness columns, parent binding is checked by static global multiset constraints, and the binary resolution relation is checked by row-local constraints. This design gives a direct non-interactive constraint system for the UNSAT certificate at the cost of working with exact resolution rather than the weak-resolution polynomial encoding of ZKUNSAT.

Resolution and DRAT Proofs. Resolution proofs provide a simple, monotone proof system for propositional unsatisfiability: each derived clause names two earlier parent clauses and a pivot variable, and checking a step reduces to validating one binary resolution inference. This regular structure is well suited to trace-based arithmetization, because parent binding can be expressed as a global multiset relation and each resolution inference can be checked by row-local constraints. The drawback is proof size. Classical proof-complexity results show that some formulas, including encodings of the pigeonhole principle, require exponentially large resolution refutations [14].

Modern SAT solvers therefore often emit richer clausal proof logs, most notably DRAT proofs [26, 31]. A DRAT proof is a sequence of clause additions and deletions. Deletions keep the active clause database small, while additions are justified by redundancy conditions such as reverse unit propagation (RUP) or resolution asymmetric tautology (RAT), rather than by naming two explicit resolution parents. This makes DRAT more expressive than plain resolution and compatible with many solver optimizations; in particular, DRAT is closely related to stronger proof systems such as extended resolution [16].

Despite these advantages, DRAT is less attractive as the first target for our AIR backend. Deletion steps make the clause database time-dependent: a later parent reference must prove not only that the referenced clause was derived, but also that it remains active at the point of use. Addition steps require checking the RAT condition against the current active database, which involves clauses containing the negated pivot and propagation-style redundancy checks. Encoding these requirements would introduce dynamic occurrence queries, active-set bookkeeping, and propagation traces that are substantially less row-local than binary resolution. Thus, although DRAT may reduce the number or width of derived clauses, its heterogeneous verifier can add enough constraint complexity to erode that advantage in a zero-knowledge backend. ZK-ProVer therefore uses resolution proofs as the initial UNSAT certificate format; supporting DRAT-style proof logs is a natural extension once the additional database-liveness and RAT checking constraints can be encoded efficiently.

ZK for Formal Methods. Several recent works study zero-knowledge certification for logical formulas and solver results. ZKSMT [19] extends zero-knowledge certification from Boolean formulas to SMT validity proofs by designing a VM specialized for SMT proof verification. It targets theorem validation for supported SMT fragments, whereas our protocol targets SAT-based program verification: Phase I validates the frontend-to-CNF translation and binds the resulting bit-precise CNF formula; Phase II certifies either a satisfying assignment or a resolution proof for that same formula.

Another related direction is zero-knowledge certification for proof-assistant derivations. zkPi [18] proves the existence of Lean proofs in zero knowledge, thereby moving ZKPs from program execution and circuit satisfiability toward machine-checked theorem proving.

Zero-knowledge techniques have also been applied to adjacent formal-methods tasks. Fang et al. [9] introduce zero-knowledge static program analysis, allowing a prover to certify analysis results without revealing sensitive program details. More recently, ZK-CEC [28] studies zero-knowledge circuit functional equivalence for hardware verification, where the goal is to prove equivalence between a secret design and a public specification.

6 Discussion

The current evaluation focuses on C programs from the supported SV-COMP slice with acyclic or shallow bounded control structure, in particular programs without deep loops or complex branching. This scope should be understood as a deliberate component-level evaluation rather than as a claim that the current prototype already covers the full range of SAT-based software verification. It also reflects a more fundamental scalability limitation of SAT-based bounded verification [5, 7]. When the program state space grows, fully unfolding the transition relation and error-state condition into a SAT instance can cause the CNF formula to grow rapidly, and in the worst case exponentially with the exploration bound. Programs with few source lines may still become difficult to handle when the loop bound is high or when the control structure contains deeply nested branches or recursive behavior. On the UNSAT side, the corresponding resolution refutation may then grow beyond practical prover limits, even if the final zero-knowledge proof remains succinct for the verifier.

This limitation suggests that zero-knowledge certification should not be tied exclusively to fully unfolded CNF formulas. A broader design space is to move the certification relation closer to a transition-system view of program verification. Intermediate formats such as BTOR2 provide an explicit word-level representation for bit-vector and array transition systems [22]. Rather than expanding the program state space and its error assertions into one large CNF formula, the verifier relation could treat the program as a finite-state machine. In the SAT case, the private witness would be a bounded execution trace of this machine that reaches an error state, and the zero-knowledge proof would certify the validity of that trace without revealing the concrete inputs or intermediate states. In the UNSAT case, instead of proving a large resolution refutation for the fully unfolded formula, the prover could certify an invariant showing that the error states are unreachable, following the invariant-based view of IC3/PDR-style safety verification [3, 6]. Such an invariant-based formulation would better match verification problems with larger or more structured state spaces, and it frames ZK-ProVer as a first step toward privacy-preserving certification across multiple abstraction levels of program verification.

7 Conclusion

We presented a two-phase protocol for privacy-preserving certification of SAT-based program verification. Phase I validates the deterministic frontend-to-CNF translation from the source program and assertions and commits to the resulting SAT formula, while Phase II certifies the solver result for that committed formula. The SAT branch avoids disclosing concrete counterexamples, and the UNSAT branch avoids transmitting and repeatedly checking the full resolution proof. This structure allows the verifier to learn the verification result through reusable zero-knowledge proofs linked to the committed formula.

The current implementation realizes the Phase II AIRs over a Plonky3-based STARK backend and evaluates the two phases separately. On the supported SV-COMP slice, Phase I shows that bounded verification-condition translations

can be checked once and reused through compact commitments. On UNSAT benchmarks, ZK-ProVer achieves an $11.1\times$ geometric-mean verifier speedup over ZKUNSAT and a $410.4\times$ geometric-mean reduction in verifier-side communication. Together, these results suggest that zero-knowledge certification can be a practical layer on top of existing SAT-based verification workflows.

References

- [1] Thomas Attema, Serge Fehr, and Michael Klooß. 2023. Fiat–Shamir Transformation of Multi-Round Interactive Proofs. *Journal of Cryptology* 36, 4 (2023), 36. doi:10.1007/s00145-023-09478-y
- [2] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2018. Scalable, Transparent, and Post-Quantum Secure Computational Integrity. Cryptology ePrint Archive, Paper 2018/046. <https://eprint.iacr.org/2018/046>
- [3] Dirk Beyer and Matthias Dangl. 2020. Software Verification with PDR: An Implementation of the State of the Art. In *Tools and Algorithms for the Construction and Analysis of Systems (Lecture Notes in Computer Science, Vol. 12078)*. Springer, 3–21. doi:10.1007/978-3-030-45190-5_1
- [4] Armin Biere. 2008. PicoSAT Essentials. *Journal on Satisfiability, Boolean Modeling and Computation* 4 (2008), 75–97. doi:10.3233/SAT190039
- [5] Armin Biere, Alessandro Cimatti, Edmund M. Clarke, Masahiro Fujita, and Yunshan Zhu. 1999. Symbolic model checking using SAT procedures instead of BDDs. In *Proceedings of the 36th Annual Design Automation Conference*. ACM, 317–320. doi:10.1145/309847.309942
- [6] Aaron R. Bradley. 2011. SAT-Based Model Checking without Unrolling. In *Verification, Model Checking, and Abstract Interpretation (Lecture Notes in Computer Science, Vol. 6538)*. Springer, 70–87. doi:10.1007/978-3-642-18275-4_7
- [7] Edmund Clarke, Daniel Kroening, and Flavio Lerda. 2004. A Tool for Checking ANSI-C Programs. In *Tools and Algorithms for the Construction and Analysis of Systems (Lecture Notes in Computer Science, Vol. 2988)*. Springer, 168–176. doi:10.1007/978-3-540-24730-2_15
- [8] Tim Dokchitser and Alexandr Bulkin. 2023. Zero Knowledge Virtual Machine step by step. Cryptology ePrint Archive, Paper 2023/1032. <https://eprint.iacr.org/2023/1032>
- [9] Zhiyong Fang, David Darais, Joe Near, and Yupeng Zhang. 2021. Zero Knowledge Static Program Analysis. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2271–2288. <https://david.darais.com/assets/papers/zk-sa/zk-sa.pdf>
- [10] Amos Fiat and Adi Shamir. 1987. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology – CRYPTO ’86 (Lecture Notes in Computer Science, Vol. 263)*. Springer, 186–194. doi:10.1007/3-540-47721-7_12
- [11] Oded Goldreich, Silvio Micali, and Avi Wigderson. 1991. Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems. *J. ACM* 38, 3 (1991), 690–728. doi:10.1145/116825.116852
- [12] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schafneggger. 2019. Poseidon: A New Hash Function for Zero-Knowledge Proof Systems. Cryptology ePrint Archive, Paper 2019/458. <https://eprint.iacr.org/2019/458>
- [13] Ulrich Haböck. 2022. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Paper 2022/1530. <https://eprint.iacr.org/2022/1530>
- [14] Armin Haken. 1985. The Intractability of Resolution. *Theoretical Computer Science* 39 (1985), 297–308. doi:10.1016/0304-3975(85)90144-6
- [15] Marijn J. H. Heule, Oliver Kullmann, and Victor W. Marek. 2016. Solving and Verifying the Boolean Pythagorean Triples Problem via Cube-and-Conquer. In *Theory and Applications of Satisfiability Testing – SAT 2016 (Lecture Notes in Computer Science, Vol. 9710)*. Springer, 228–245. doi:10.1007/978-3-319-40970-2_15
- [16] Benjamin Kiesl, Adrián Rebola-Pardo, and Marijn J. H. Heule. 2018. Extended Resolution Simulates DRAT. In *Automated Reasoning – 9th International Joint Conference, IJCAR 2018 (Lecture Notes in Computer Science, Vol. 10900)*. Springer, 516–531. doi:10.1007/978-3-319-94205-6_34
- [17] James C. King. 1976. Symbolic Execution and Program Testing. *Commun. ACM* 19, 7 (1976), 385–394.
- [18] Evan Laufer, Alex Ozdemir, and Dan Boneh. 2024. zkPi: Proving Lean Theorems in Zero-Knowledge. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 4301–4315. doi:10.1145/3658644.3670322
- [19] Daniel Luick, John C Kolesar, Timos Antonopoulos, William R Harris, James Parker, Ruzica Piskac, Eran Tromer, Xiao Wang, and Ning Luo. 2024. ZKSMT: A VM for Proving SMT Theorems in Zero Knowledge. In *33rd USENIX Security Symposium (USENIX Security 24)*. 3837–3845.
- [20] Ning Luo, Timos Antonopoulos, William Harris, Ruzica Piskac, Eran Tromer, and Xiao Wang. 2022. Proving UNSAT in Zero Knowledge. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2203–2217. doi:10.1145/3548606.3559373
- [21] Ralph C. Merkle. 1988. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology – CRYPTO ’87 (Lecture Notes in Computer Science, Vol. 293)*. Springer, 369–378. doi:10.1007/3-540-48184-2_32
- [22] Aina Niemetz, Mathias Preiner, Clifford Wolf, and Armin Biere. 2018. Btor2, BtorMC and Boolector 3.0. In *Computer Aided Verification (Lecture Notes in Computer Science, Vol. 10981)*. Springer, 587–595. doi:10.1007/978-3-319-96145-3_32
- [23] Shahar Papini and Ulrich Haböck. 2023. Improving logarithmic derivative lookups using GKR. Cryptology ePrint Archive, Paper 2023/1284. <https://eprint.iacr.org/2023/1284>
- [24] Plonky3 Contributors. 2026. Plonky3: A Toolkit for Polynomial IOPs. <https://github.com/Plonky3/Plonky3>. Accessed: 2026-04-25.
- [25] A. Rao and Y. Wang. 2024. Formal Verification of Autonomous Vehicles: Bridging the Gap between Model-Based Design and Model Checking. *SAE International Journal of Advances and Current Practices in Mobility* 6, 2 (2024), 814–826. doi:10.4271/2023-01-0116

- [26] Adrián Rebola-Pardo and Luis Cruz-Filipe. 2018. Complete and Efficient DRAT Proof Checking. In *Proceedings of the 18th Conference on Formal Methods in Computer-Aided Design*. IEEE, 197–205. doi:10.23919/FMCAD.2018.8602993
- [27] Suneel Sarswat and Abhishek Kr Singh. 2020. Formally Verified Trades in Financial Markets. In *Formal Methods and Software Engineering (Lecture Notes in Computer Science, Vol. 12531)*, Shang-Wei Lin, Zhe Hou, and Brendan Mahony (Eds.). Springer, 217–232. doi:10.1007/978-3-030-63406-3_13
- [28] Sirui Shen, Zunchen Huang, and Chenglu Jin. 2026. Proving Circuit Functional Equivalence in Zero Knowledge. arXiv preprint arXiv:2601.11173. doi:10.48550/arXiv.2601.11173
- [29] Succinct Labs. 2026. SP1: A Zero-Knowledge Virtual Machine for RISC-V Programs. <https://github.com/succinctlabs/sp1>. Version 6.0.2; accessed: 2026-04-26.
- [30] Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. 2016. EMP-toolkit: Efficient MultiParty Computation Toolkit. <https://github.com/emp-toolkit>.
- [31] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt Jr. 2014. DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs. In *Theory and Applications of Satisfiability Testing – SAT 2014 (Lecture Notes in Computer Science, Vol. 8561)*. Springer, 422–429. https://link.springer.com/chapter/10.1007/978-3-319-09284-3_31
- [32] Lintao Zhang and Sharad Malik. 2003. Validating SAT solvers using an independent resolution-based checker: Practical implementations and other applications. In *Proceedings of the Design, Automation and Test in Europe Conference (DATE)*. IEEE, 10880–10885. https://www.princeton.edu/~chaff/publication/DATE2003_validating_sat_solver.pdf
- [33] Yan Zhang, Sheng Wen, and Cheng Xu. 2018. Blockchain-Based Certificate Verification System Using Smart Contracts. *International Journal of Advanced Computer Science and Applications* 9, 8 (2018), 166–171. <https://doi.org/10.14569/IJACSA.2018.090822>